Project Proposal Form

Institute of Information Technology (IIT)

University of Dhaka

Student's Name:	**Pronob Karmoker**		
Student's Roll:	**1431**	**Phone:**	**01745330624**

SBLLM-Optimizer: A Search-Based LLM-Powered VS Code Extension for Intelligent Code Optimization

Project Description

Modern software often contains inefficient code that leads to performance issues and resource wastage. Detecting and optimizing such code manually can be difficult and time-consuming. SBLLM-Optimizer is a VS Code extension inspired by the paper "Search-Based LLMs for Code Optimization (SBLLM)" that helps developers automatically optimize inefficient code using Large Language Models (LLMs) and search-based iterative refinement. The system analyzes code, generates optimized alternatives, evaluates execution performance, and progressively improves solutions directly inside VS Code.

Problem Statement & Objectives

Writing efficient and optimized code is a challenging and time-consuming task for developers. Most programmers focus on code correctness rather than execution efficiency, which often leads to performance issues, resource wastage, and inefficient software systems. Existing optimization tools either rely on limited rule-based methods or use one-step AI generation approaches that fail to discover deeper optimization opportunities.

The main objective of SBLLM-Optimizer is to develop an AI-powered VS Code extension that automatically detects inefficient code, generates optimized alternatives, and iteratively improves code performance using Large Language Models (LLMs) and search-based optimization techniques. The system aims to help developers write faster and more efficient code directly inside the development environment.

Proposed Solution

- **Intelligent Code Analysis:** Analyze source code inside VS Code and detect inefficient patterns such as nested loops, redundant computations, and suboptimal algorithms.
- **LLM-based Optimization Generation:** Use Large Language Models (LLMs) to generate multiple optimized versions of inefficient code snippets.
- **Execution-based Evaluation:** Benchmark generated code using execution time, correctness, and speedup metrics to evaluate optimization quality.
- **Search-based Iterative Refinement:** Iteratively improve generated solutions using execution feedback and representative optimization samples inspired by the SBLLM framework.
- **Optimization Pattern Retrieval:** Retrieve relevant optimization patterns from existing examples and inject them into LLM prompts to guide better optimization.
- **Genetic Operator-inspired Reasoning:** Apply crossover and mutation-inspired prompting strategies to combine optimization ideas and discover improved solutions.
- **VS Code Integration:** Display optimized code, execution comparisons, and optimization explanations directly inside VS Code through an interactive interface.
- ns will be displayed directly inside VS Code using an interactive user interface.

Key Technologies & Tools

Language: Python ,JavaScript , Shell
AI & LLM Integration : OpenAI API / Gemini API / Ollama , Prompt Engineering
VS Code Development : VS Code Extension API , Node.js
Data Storage & Retrieval : JSON / SQLite
Version control: Git, Github

References:

[1] Shuzheng Gao, Cuiyun Gao, Wenchao Gu, Michael R. Lyu, “Search-Based LLMs for Code Optimization,” arXiv preprint arXiv:2408.12159, 2024.

Supervisor’s Name: Dr Md. Nurul Ahad Tawhid

Signature of the supervisor:

Date: 18-05-2026

Proposal Presentation Feedback:

Midterm Presentation Feedback: